Light

Programming — Testing & Robustness

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub
Long-Answer
Notes Index
What you must know
Toolkit
1. Testing
2. Defensive design
3. Maintainability
Worked examples
Quick reference
Typical pitfalls

OCR GCSE Computer Science — **Producing Robust Programs (J277/02 Section 2.3)**. This topic covers defensive design techniques (validation, authentication, input sanitisation), testing strategies and test data types, identifying and fixing errors, and writing maintainable code.

Exam questions typically ask you to **select appropriate test data**, **identify error types**, **write validation code**, and **explain how to make programs more robust**. The 2024 exam included questions on choosing test data for a given range and identifying the type of error in code.

What you must know

This checklist covers the key exam-visible concepts. You should be able to explain each point and apply it in programming contexts.

- **Describe** the purpose of testing and the difference between iterative and final testing.
- **Select** appropriate test data: normal, boundary, and erroneous (invalid) data.
- **Identify** and distinguish between syntax errors, logic errors, and runtime errors.
- **Explain** validation techniques: range check, type check, length check, presence check, format check.
- **Write** code to implement validation using loops and conditions.

- **Describe** authentication methods and their purpose.
- **Explain** input sanitisation and why it is important.
- **Apply** maintainability techniques: comments, indentation, meaningful identifiers.

> **Specification note.** OCR J277 Section 2.3 requires understanding of defensive design, testing, and maintainability. Questions often give you a scenario (e.g., "age must be between 0 and 120") and ask you to select test data or write validation code. You must know all five validation types and all three error types.

Toolkit

The three types of test data

When testing a program, you should use three types of test data to ensure it handles all situations correctly:

Type	Definition	Purpose	Example (age 0-120)
Normal	Typical, expected data within the valid range	Check the program works with everyday inputs	25, 45, 70
Boundary	Data at the edges of the valid range	Check the program handles limits correctly	0, 1, 119, 120
Erroneous	Invalid data that should be rejected	Check the program doesn't accept bad data	-5, 150, "abc"

> **Exam tip — boundary data.**
> For a range like 0-120, boundary values are: **0** (lower valid), **120** (upper valid), and optionally **-1** and **121** (just outside). Always include both ends of the valid range.

The three types of errors

Error type	When detected	Cause	Example
Syntax error	Before running (won't compile/run)	Breaking the rules of the language	Missing bracket, misspelled keyword
Logic error	Program runs but gives wrong results	Mistake in the algorithm/logic	Using + instead of -, wrong condition

Error type	When detected	Cause	Example
Runtime error	During execution (program crashes)	Something impossible happens	Division by zero, file not found

The five validation checks

Check	What it validates	Example use
Range check	Value is within a specified range	Age between 0 and 120
Type check	Data is the correct data type	Age is an integer, not text
Length check	Input has correct number of characters	Password at least 8 characters
Presence check	Required field is not empty	Username field must be filled in
Format check	Data matches a required pattern	Email contains @ symbol, date is DD/MM/YYYY

Exam tip — memorise all five.
A common question gives a scenario and asks which validation check should be used. Remember: **R**ange, **T**ype, **L**ength, **P**resence, **F**ormat — "Real Teachers Love Perfect Formatting".

Iterative vs Final testing

Iterative testing

- Testing during development
- Test each module/section as it's written
- Fix errors before moving on
- Continuous process throughout development

Final (terminal) testing

- Testing at the end of development
- Test the complete program
- Check all features work together
- Use a full test plan with all test data types

Authentication vs Validation

Validation

- Checks data is **reasonable and sensible**
- Is the age a number between 0-120?
- Is the email in the correct format?
- Prevents invalid data entering the system

Authentication

- Checks the user is **who they claim to be**
- Username and password
- Biometrics (fingerprint, face recognition)
- Two-factor authentication (2FA)

Key difference: Validation checks the **data** is correct. Authentication checks the **user** is correct.

1. Testing

Testing is essential to ensure a program works correctly and handles all possible inputs appropriately. A good test plan uses a variety of test data to check both normal operation and edge cases.

Why we test

- To find and fix **errors** (bugs) before release
- To check the program meets its **requirements**
- To ensure the program handles **unexpected input** gracefully
- To verify **all paths** through the code work correctly

Creating a test plan

A test plan documents the tests to be performed:

- **Test number** — unique identifier
- **Description** — what is being tested
- **Test data** — the input values
- **Expected result** — what should happen
- **Actual result** — what actually happened
- **Pass/Fail** — did it work correctly?

Selecting test data

For any input with constraints, you should test with all three types of data. Consider a program that accepts a percentage mark (0-100):

Test	Data type	Test value	Expected result	Reason
1	Normal	50	Accepted	Typical value in middle of range
2	Normal	75	Accepted	Another typical value
3	Boundary	0	Accepted	Lower boundary (valid)

Test	Data type	Test value	Expected result	Reason
4	Boundary	100	Accepted	Upper boundary (valid)
5	Erroneous	-1	Rejected	Just below lower boundary
6	Erroneous	101	Rejected	Just above upper boundary
7	Erroneous	"abc"	Rejected	Wrong data type (text not number)

Types of errors

Syntax errors

The code breaks the rules of the programming language. The program **won't run at all**.

```
// Syntax error - missing bracket print("Hello" // Syntax error - misspelled keyword prnit("Hello")
```

Logic errors

The code runs but produces **wrong results**. The algorithm is incorrect.

```
// Logic error - wrong operator average = total * count // Should be: total / count // Logic error - wrong condition if age > 18 then // Should be: age >= 18
```

Runtime errors

The code runs but **crashes** when something impossible is attempted.

```
// Runtime error - division by zero result = 100 / 0 // Runtime error - file not found open("missing.txt") // Runtime error - array out of bounds data[100] // if array has only 10 items
```

Worked Example 1 — Selecting test data [2024 exam style]

Question: A program accepts a quantity of items to order. The quantity must be between 1 and 99 inclusive. Give one example of each type of test data and explain why you chose it. [6 marks]

Normal data: 50 [2 marks]
This is a typical value that falls within the valid range [1 mark]. It tests that the program accepts everyday, expected inputs [1 mark].
Boundary data: 1 or 99 [2 marks]

These are at the edges of the valid range [1 mark]. They test that the program correctly handles the limits of acceptable values [1 mark].
Erroneous data: 0, 100, or "five" [2 marks]
These are outside the valid range or wrong data type [1 mark]. They test that the program correctly rejects invalid inputs [1 mark].

Exam tip: Always give a **specific value** AND **explain why**. Just saying "50" without explanation loses marks.

Worked Example 2 — Identifying error types

Question: Identify the type of error in each code snippet. [3 marks]

```
// Snippet A area = length * width print("Area is: " area) //
Snippet B discount = price * 0.1 finalPrice = price * discount
// intended to subtract // Snippet C numbers = [1, 2, 3]
print(numbers[5])
```

Snippet A: Syntax error [1 mark]
Missing concatenation operator (+) between the string and the variable. The program won't run.
Snippet B: Logic error [1 mark]
Uses multiplication (*) instead of subtraction (-). The program runs but gives wrong results.
Snippet C: Runtime error [1 mark]
Tries to access index 5 of an array with only 3 elements (indices 0, 1, 2). Crashes during execution.

Exam tip: Ask yourself: (1) Would it run? No = syntax. (2) Does it crash? Yes = runtime. (3) Wrong answer? = logic.

Worked Example 3 — Creating a test plan

Question: A program calculates postage cost. Items under 100g cost £1.50, items 100g-500g cost £3.00, items over 500g cost £5.00. Create a test plan with at least 5 tests. [5 marks]

Test	Data	Type	Expected output
1	50g	Normal	£1.50
2	250g	Normal	£3.00

Test	Data	Type	Expected output
3	100g	Boundary	£3.00
4	500g	Boundary	£3.00
5	-10g	Erroneous	Error message

Exam tip: Include tests for each category/band in the program, plus boundary values between categories.

Worked Example 4 — Iterative vs final testing

Question: Explain the difference between iterative testing and final testing. [4 marks]

Iterative testing is carried out during development [1 mark]. Each module or section is tested as it is written, and errors are fixed before moving on to the next part [1 mark].
Final testing is carried out after the program is complete [1 mark]. The entire program is tested as a whole using a comprehensive test plan to check all features work together correctly [1 mark].

Exam tip: Both types are needed — iterative testing catches errors early; final testing ensures everything works together.

2. Defensive design

Defensive design means anticipating things that could go wrong and writing code to handle them. A robust program doesn't crash when given unexpected input — it handles errors gracefully and guides users to provide correct data.

Why use defensive design?

- Prevents crashes from unexpected input
- Protects data integrity
- Improves user experience with helpful error messages
- Prevents security vulnerabilities
- Makes programs more reliable

Key defensive techniques

- **Validation** — checking data is sensible before using it
- **Authentication** — verifying user identity
- **Input sanitisation** — cleaning input to remove dangerous content

- **Error handling** — catching and managing errors gracefully

Validation techniques in code

Here's how to implement each of the five validation checks in OCR Exam Reference Language:

Range check

```
// Check age is between 0 and 120 valid = false while valid == false age = input("Enter age: ") if age >= 0 AND age <= 120 then valid = true else print("Age must be 0-120") endif endwhile
```

Length check

```
// Check password is at least 8 characters valid = false while valid == false password = input("Enter password: ") if LEN(password) >= 8 then valid = true else print("Password must be at least 8 characters") endif endwhile
```

Presence check

```
// Check name is not empty valid = false while valid == false name = input("Enter name: ") if LEN(name) > 0 then valid = true else print("Name cannot be empty") endif endwhile
```

Format check

```
// Check email contains @ symbol valid = false while valid == false email = input("Enter email: ") if "@" IN email then valid = true else print("Email must contain @") endif endwhile
```

Authentication

Authentication verifies that a user is who they claim to be. Common methods include:

Username and password

- Most common method
- Something the user knows
- Passwords should be stored encrypted
- Should enforce strong password rules

Biometrics

- Fingerprint scanning
- Face recognition
- Iris scanning
- Something the user is

Two-factor authentication (2FA)

- Combines two methods
- e.g., password + code sent to phone
- More secure than single factor
- Something you know + have

```
// Simple authentication example storedUsername = "admin"
storedPassword = "secret123" username = input("Enter username: ")
password = input("Enter password: ") if username == storedUsername
AND password == storedPassword then print("Access granted") else
print("Access denied") endif
```

Input sanitisation

Input sanitisation removes or escapes potentially dangerous characters from user input before it is processed or stored. This protects against attacks like SQL injection.

Why sanitise input?

- Prevent SQL injection attacks
- Prevent cross-site scripting (XSS)
- Remove unwanted characters
- Ensure data consistency

Example dangerous input:

```
// User enters this as their name: "; DROP TABLE users; --" //
Without sanitisation, this could // delete the entire database!
```

Worked Example 5 — Writing validation code [2024 exam style]

Question: Write an algorithm that asks the user to enter a month number (1-12) and keeps asking until a valid number is entered. [4 marks]

```
valid = false while valid == false month = input("Enter
month (1-12): ") if month >= 1 AND month <= 12 then valid =
true else print("Invalid. Enter 1-12.") endif endwhile
```

Marking:

- Loop that repeats until valid [1 mark]
- Input statement [1 mark]
- Correct condition (>= 1 AND <= 12) [1 mark]
- Error message and loop continues if invalid [1 mark]

> **Exam tip:** The pattern is always: set valid to false, WHILE not valid, get input, IF valid THEN set valid to true, ELSE show error.

Worked Example 6 — Identifying validation types

Question: State which type of validation check should be used for each of the following. [4 marks]

- a) A postcode must be 6-8 characters long
- b) A score must be between 0 and 100
- c) A date must be in the format DD/MM/YYYY
- d) A customer must enter their phone number

- a) **Length check** [1 mark] — checking the number of characters
- b) **Range check** [1 mark] — checking value is within a range
- c) **Format check** [1 mark] — checking data matches a pattern
- d) **Presence check** [1 mark] — checking field is not empty

> **Exam tip:** Look for keywords: "between X and Y" = range, "characters long" = length, "format/pattern" = format, "must enter" = presence.

Worked Example 7 – Authentication scenario

Question: A bank wants to improve the security of its online banking. Describe two authentication methods they could use and explain why using two methods together is more secure than one. [6 marks]

Method 1: Password [2 marks]
The user enters a secret password that only they know [1 mark]. This should be strong (mixed case, numbers, symbols) and stored encrypted [1 mark].
Method 2: One-time code sent to phone [2 marks]
After entering the password, a code is sent via SMS to the user's registered phone [1 mark]. The user must enter this code to complete login [1 mark].
Why two is more secure: [2 marks]
Using two factors (something you know + something you have) means an attacker would need to steal both the password AND the phone [1 mark]. If only the password is compromised, the attacker still cannot access the account [1 mark].

> **Exam tip:** The three factors are: something you **know** (password), something you **have** (phone/card), something you **are** (biometrics).

Worked Example 8 – Input sanitisation explanation

Question: Explain what input sanitisation is and why it is important. [3 marks]

Input sanitisation is the process of checking and cleaning user input to remove or escape potentially harmful characters [1 mark].
It is important because malicious users could enter code (such as SQL commands) into input fields [1 mark].

Without sanitisation, this malicious input could be executed by the system, allowing attackers to steal data, delete records, or take control of the system [1 mark].

Exam tip: You don't need to write sanitisation code at GCSE, but you must understand what it is and why it's needed.

3. Maintainability

Maintainable code is easy to read, understand, and modify. This is important because code often needs to be updated by different programmers over time. Good maintainability practices save time and reduce errors.

Why maintainability matters

- Other programmers may need to modify your code
- You may return to code months later and forget how it works
- Bugs are easier to find in well-organised code
- Features are easier to add to clear, structured code
- Professional standard in industry

The three key techniques

1. **Comments** — explain what the code does and why
2. **Indentation** — show code structure visually
3. **Meaningful identifiers** — use descriptive variable and function names

Using comments effectively

Comments explain the purpose and logic of code. They are ignored by the computer but invaluable for humans.

Good commenting:

```
// Calculate the total price including 20% VAT totalPrice =
basePrice * 1.2 // Check if user is old enough to vote (18+) if age
>= 18 then canVote = true endif
```

Poor commenting (too obvious):

```
// Set x to 5 x = 5 // Add 1 to counter counter = counter + 1 //
Print the message print(message)
```

Exam tip — what makes a good comment?
Good comments explain **WHY** or **WHAT** at a higher level, not just repeat what the code obviously does. "Add 1 to counter" is useless. "Track number of failed login attempts" is useful.

Using indentation

Indentation shows the structure of code — which statements are inside loops, conditions, or functions.

Well indented (correct):

```
for i = 1 to 10 if i MOD 2 == 0 then print(i) endif next i
```

Poorly indented (hard to read):

```
for i = 1 to 10 if i MOD 2 == 0 then print(i) endif next i
```

Using meaningful identifiers

Variable and function names should describe what they contain or do. This makes code self-documenting.

Good identifiers:

```
totalPrice = 0 customerName = "John" numberOfItems = 5 isLoggedIn =
true calculateDiscount() validateEmail()
```

Poor identifiers:

```
tp = 0 n = "John" x = 5 flag = true calc() check()
```

Naming conventions.

Common conventions include:

- **camelCase** — firstName, totalPrice (common in JavaScript, Java)
- **snake_case** — first_name, total_price (common in Python)
- **PascalCase** — FirstName, TotalPrice (common for class names)

The key is consistency — pick one style and stick with it.

Worked Example 9 – Improving code maintainability

Question: The following code works but is poorly written. Rewrite it to improve maintainability. [4 marks]

```
x = 0 for i = 1 to 10 y = input("?") x = x + y next i z = x / 10
print(z)
```

```
// Calculate the average of 10 numbers entered by the user
total = 0 for count = 1 to 10 number = input("Enter number:
```

```
") total = total + number next count average = total / 10
print("Average: " + average)
```

Improvements made:

- Added comment explaining purpose [1 mark]
- Changed x to "total", y to "number", z to "average" [1 mark]
- Added proper indentation inside the loop [1 mark]
- Improved input prompt and output message [1 mark]

Worked Example 10 — Explaining maintainability benefits

Question: Explain three ways a programmer can make their code more maintainable and why each is important. [6 marks]

1. Use comments [2 marks]
Comments explain what the code does and why [1 mark]. This helps other programmers (or yourself later) understand the purpose without having to analyse every line [1 mark].
2. Use meaningful variable names [2 marks]
Names like "totalPrice" describe what data is stored [1 mark]. This makes code self-documenting and easier to follow than single letters like "x" [1 mark].
3. Use consistent indentation [2 marks]
Indentation shows the structure of the code visually [1 mark]. It makes it clear which statements are inside loops or conditions, making errors easier to spot [1 mark].

Worked Example 11 — Adding comments to code

Question: Add appropriate comments to the following code. [3 marks]

```
password = input("Enter password: ") while LEN(password) < 8
print("Too short") password = input("Enter password: ") endwhile
```

```
// Get password from user and validate minimum length
password = input("Enter password: ") // Keep asking until
password is at least 8 characters while LEN(password) < 8
print("Too short") password = input("Enter password: ")
endwhile
```

Marking: 1 mark for each useful comment that explains purpose/logic (not just restating the code).

Exam tip: Don't write "Get password" — that's obvious. Write "Validate minimum length of 8 characters for security" — that explains WHY.

Worked Example 12 – Complete robust input routine

Question: Write a complete, well-commented algorithm that asks the user for their age. The age must be a whole number between 0 and 120. The program should keep asking until a valid age is entered, then display "Valid age entered". [6 marks]

```
// Input validation for user age // Age must be between 0
and 120 inclusive valid = false // Keep asking until valid
input received while valid == false // Get age from user age
= input("Enter your age (0-120): ") // Check if age is
within valid range if age >= 0 AND age <= 120 then valid =
true else print("Invalid age. Please enter 0-120.") endif
endwhile print("Valid age entered")
```

Marking:

- Correct loop structure (while valid == false) [1 mark]
- Correct input statement [1 mark]
- Correct range check condition (>= 0 AND <= 120) [1 mark]
- Sets valid to true when correct / error message when wrong [1 mark]
- Appropriate comments [1 mark]
- Meaningful identifier names and indentation [1 mark]

Summary of worked examples

#	Topic	Question type	Key skill
1	Test data selection	Give examples + explain	Normal, boundary, erroneous
2	Error types	Identify from code	Syntax, logic, runtime
3	Test plan	Create test table	All data types + expected results
4	Testing types	Explain difference	Iterative vs final
5	Validation code	Write algorithm	Range check with loop
6	Validation types	Match to scenario	All five validation checks
7	Authentication	Describe methods	Two-factor authentication
8	Input sanitisation	Explain concept	Why sanitisation is needed

#	Topic	Question type	Key skill
9	Improving code	Rewrite code	Apply all three techniques
10	Maintainability	Explain benefits	Comments, names, indentation
11	Adding comments	Annotate code	Useful vs obvious comments
12	Complete routine	Write full algorithm	Validation + maintainability

Quick reference

Key terms

Term	Definition (exam-answer style)
Normal data	Typical, expected data that falls within the valid range and should be accepted
Boundary data	Data at the extreme edges of the valid range, used to check limits are handled correctly
Erroneous data	Invalid data that should be rejected by the program (outside range or wrong type)
Syntax error	An error where the code breaks the rules of the programming language; program won't run
Logic error	An error where the code runs but produces incorrect results due to faulty algorithm
Runtime error	An error that occurs during program execution, causing it to crash
Validation	Checking that input data is reasonable, sensible, and within expected bounds
Authentication	Verifying that a user is who they claim to be (e.g., using password or biometrics)
Input sanitisation	Removing or escaping potentially dangerous characters from user input
Maintainability	How easy code is to read, understand, and modify by programmers

Validation checks at a glance

Check	Checks that...	Example scenario
Range	Value is within min-max bounds	Age 0-120, mark 0-100
Type	Data is correct type	Age is integer not text

Check	Checks that...	Example scenario
Length	Correct number of characters	Password 8+ chars, postcode 6-8
Presence	Field is not empty	Name must be entered
Format	Matches required pattern	Email has @, date is DD/MM/YYYY

Error identification flowchart

To identify error type, ask:

1. **Does the program run at all?**
   No → **Syntax error** (code breaks language rules)
2. **Does the program crash during execution?**
   Yes → **Runtime error** (impossible operation)
3. **Does the program give wrong results?**
   Yes → **Logic error** (algorithm is wrong)

Typical pitfalls

Pitfall 1 — Confusing boundary and erroneous.
Wrong: Saying -1 is boundary data for range 0-100.
Right: -1 is **erroneous** (outside range). **Boundary** = 0 and 100 (the valid edges).

Pitfall 2 — Only testing one boundary.
Wrong: Testing only 0 for range 0-100.
Right: Test **both** boundaries: 0 (lower) AND 100 (upper).

Pitfall 3 — Confusing validation and authentication.
Wrong: "Validation checks the user is allowed access."
Right: Validation checks data is sensible. **Authentication** checks user identity.

Pitfall 4 — Confusing syntax and logic errors.
Wrong: "The program gives wrong answer, so it's a syntax error."
Right: Wrong answer = **logic error**. Syntax errors stop the program running at all.

Pitfall 5 — Writing obvious comments.
Wrong: "// Add 1 to x" for line: x = x + 1
Right: Comments should explain **purpose**, e.g., "// Count number of items processed"

Pitfall 6 — Not giving specific test values.

Wrong: "Use normal data to test."
Right: Give a **specific value**: "Use 50 as normal data because it's a typical value within the range."

Pitfall 7 — Incomplete validation loops.
Wrong: Using IF instead of WHILE for validation.
Right: Use **WHILE** to keep asking until valid. IF only checks once.

Pitfall 8 — Wrong range check operators.
Wrong: `if age > 0 AND age < 120` for range 0-120
Right: `if age >= 0 AND age <= 120` — include the boundaries!

Pitfall 9 — Confusing length and format checks.
Wrong: "Check postcode format is 6-8 characters."
Right: Checking **number** of characters = **length** check. Format checks the **pattern** (e.g., letters then numbers).

Pitfall 10 — Forgetting runtime errors.
Wrong: Only considering syntax and logic errors.
Right: Remember **runtime errors**: division by zero, file not found, array index out of bounds.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.